

Vega-Lite: A Grammar of Interactive Graphics:

My goal is to share everything I've learned about Vega-Lite so the first steps of this journey become easier and faster for others.

One of the things that attracted me to Vega-Lite was its wide range of applications. You can use it as an embedding visualization language in Python, R, and Power BI. Since many data analysts use Power BI, this tutorial explains Vega-Lite syntax in both Power BI (Deneb) and Observable environments, along with their differences, to make learning easier and more practical.

This tutorial mainly happy to receive feedback and additional explanations from others. If you have experience with Vega-Lite or know useful information that is not included in this tutorial, we can improve this knowledge together

Basic concept of Structure:

In the following sections, we will explain the basic skeleton of Vega-Lite syntax in both Observable and Deneb.

Observable Environment :

js (JavaScript) focuses on foundational and beginner-level concepts. I probably won't cover every advanced topic, but I'll try to explain the core ideas clearly.

I would becript)

Import vega-lite in observable

```
import {vl} from "@vega/vega-lite-api"
```

```
vl.marktype()  
.data()  
.transform()  
.encode()
```

```
vl.x()  
vl.y()  
vl.color()  
vl.size()  
)  
.render()
```

Deneb(Power Bi)

```
{  
  "data":{  
    "name":dataset  
  },  
  "transform":[],  
  "params":[],  
  "mark":"type",  
  "encoding":{  
    "x":{},  
    "y":{},  
    "color":{},  
    "size":{}  
  }  
}
```

Differences in structure :

Observable	Deneb
JavaScript chaining syntax	JSON specification
.render() required	auto render
.data(data)	"dataset"
method-based	object-based

Mark (Chart Type)

Defines the type of chart.

Observable vs Deneb

Chart Type	Observable	Deneb
Scatter	<code>markPoint()</code>	<code>"mark": "point"</code>
Bar	<code>markBar()</code>	<code>"mark": "bar"</code>
Line	<code>markLine()</code>	<code>"mark": "line"</code>
Area	<code>markArea()</code>	<code>"mark": "area"</code>
Circle	<code>markCircle()</code>	<code>"mark": "circle"</code>

Params (Interaction)

Used for interaction and selection, and linking charts.

params = variables for interaction, variables for interaction (filter / brush / selection)

params =

```
- Used for:  
- filtering  
- selection  
- linking charts  
- brush (zoom)  
""  
### Observable  
  
```js  
const brush = vl.selectInterval().name("brush")

vl.markPoint()
 .params(brush)
```

```
DENEb
```

```
"params": [
 {
 "name": "brush",
 "select": {
 "type": "interval"
 }
 }
]

```

## ## Encoding (Data Mapping)

Used to map data fields to visual channels.

```
| Channel | Observable | Deneb |
|---|---|---|
| x-axis | `vl.x()` | `{ "x": {} }` |
| y-axis | `vl.y()` | `{ "y": {} }` |
| color | `vl.color()` | `{ "color": {} }` |
| size | `vl.size()` | `{ "size": {} }` |
| tooltip | `vl.tooltip()` | `{ "tooltip": {} }` |
| shape | `vl.shape()` | `{ "shape": {} }` |
| opacity | `vl.opacity()` | `{ "opacity": {} }` |

```

## ## Type Mapping

```
| Code | Meaning |
|---|---|
| Q | Quantitative (numbers) |
| N | Nominal (categories) |
| O | Ordinal (ordered categories) |
| T | Temporal (time/date) |

```

**##** Key Difference (VERY IMPORTANT)

**###** Deneb (Power BI)

- You define:

``text

field: column name

type: data type

Example:

```
"x": {
 "field": "Year",
 "type": "temporal"
}
```

---

## Observable (Vega-Lite API)

Uses method chaining:

```
vl.x().fieldT("Year")
vl.y().fieldQ("Sales")
```

---

## Core Idea

Deneb = JSON specification (explicit field + type)

Observable = function-based API (field + shorthand type)

---

## Encoding:

This section covers the core concept of Vega-Lite across both Deneb and Observable environments.

Its main purpose is to map data fields to visual encoding properties such as position, color, shape, and size.

**Source:** <https://observablehq.com/@viscourse/encodings>

**Author:** Prof. Lace Padilla

**x:**

This encoding channel controls the horizontal position of the mark, giving us a quantitative axis scale automatically.

Source: Vega-Lite documentation (Encoding Channels)

---

**y:**

The y-channel determines the vertical position of the mark. It's useful for ordinal data types too.

Source: Vega-Lite documentation (Encoding Channels)

---

**size:**

Size encoding lets us vary the size of the mark.

Source: Vega-Lite documentation (Encoding Channels)

---

**color:**

The color channel brings vibrancy to our visuals by mapping data values to colors.

Source: Vega-Lite documentation (Encoding Channels)

---

**opacity:**

Opacity encoding controls the transparency of the marks, helping us manage overplotting.

Source: Vega-Lite documentation (Encoding Channels)

---

**shape:**

Shape encoding lets us choose symbols for point marks.

Source: Vega-Lite documentation (Encoding Channels)

---

**tooltip:**

Tooltip encoding offers interactivity by providing information when hovering over a mark.

Source: Vega-Lite documentation (Encoding Channels)

---

**order:**

Order encoding influences the drawing order and point connections.

Source: Vega-Lite documentation (Encoding Channels)

---

### column & row:

These channels enable data faceting, allowing us to create multiple sub-plots within a visual.

Source: Vega-Lite documentation (Faceting)

---

## Transform in Vega-Lite (Observable & Deneb)

Transform is a core concept in Vega-Lite used to modify, filter, and reshape data before it is encoded into visual marks. It allows preprocessing steps such as filtering rows, aggregating values, creating new fields, and binning data.

---

### Filter

Filter removes rows from the dataset based on a condition.

**Purpose:** Reduce dataset before visualization.

### Observable Syntax

```
.transform([
 { filter: "datum.Year > 2010" }
])
```

### Deneb Syntax

```
"transform": [
 { "filter": "datum.Year > 2010" }
]
```

---

### Calculate

Calculate creates a new field from existing data.

**Purpose:** Feature engineering inside visualization.

## Observable Syntax

```
.transform([
 { calculate: "datum.Temp * 2", as: "Temp2" }
])
```

## Deneb Syntax

```
"transform": [
 {
 "calculate": "datum.Temp * 2",
 "as": "Temp2"
 }
]
```

---

## Aggregate

Aggregate summarizes data (mean, sum, count, etc.).

**Purpose:** Group-level analysis.

## Observable Syntax

```
.transform([
 {
 aggregate: [
 { op: "mean", field: "Temp", as: "avgTemp" }
],
 groupby: ["Year"]
 }
])
```

## Deneb Syntax

```
"transform": [
 {
 "aggregate": [
 { "op": "mean", "field": "Temp", "as": "avgTemp" }
],
 "groupby": ["Year"]
 }
]
```

---

## Bin

Bin groups continuous numeric values into ranges.

**Purpose:** Histogram-style grouping.

## Observable Syntax

```
.transform([
 { bin: true, field: "Age" }
])
```

## Deneb Syntax

```
"transform": [
 { "bin": true, "field": "Age" }
]
```

# Sort in Vega-Lite (Observable & Deneb)

Sort controls the ordering of data in visualizations. It can be applied in two main ways: simple sorting or computed (aggregate-based) sorting.

---

## 1. Simple Sort (Axis-based)

Sort categories based on their visual axis values.

## Observable

```
vl.y()
 .fieldN("disaster")
 .sort("-x")
```

## Deneb

```
"y": {
 "field": "disaster",
 "type": "nominal",
 "sort": "-x"
}
```

## Meaning

Sort categories based on the values of the x-axis (or y-axis).

---

## 2. Computed Sort (Aggregate-based)

Sort categories based on calculated values from another field.

## Observable

```
vl.y()
 .fieldN("disaster")
 .sort(vl.average("SeverityIndex").order("descending"))
```

## Deneb

```
"y": {
 "field": "disaster",
 "type": "nominal",
 "sort": {
```

```
"op": "average",
"field": "SeverityIndex",
"order": "descending"
}
}
```

## Combining Multiple Charts in One View in observable enviroment

Transform = change data

Encoding = map data to visuals

Params = interact with visuals

Layout/Composition = arrange visuals

Data

Transform

Mark + Encoding

Params (Interaction)

Layout (Size / Facet)

Composition (Combine Charts)

Render

## Data Preparation (JavaScript Layer)

This part is handled using JavaScript syntax rather than Vega-Lite. Common JavaScript features used here include:

- `const` for defining variables
- Arrow functions ( `=>` )
- Array methods such as `filter()` , `map()` , and `includes()`
- Data loading and preprocessing before visualization

The purpose of this step is to prepare and clean the data before passing it to Vega-Lite.

## Note

The Observable environment combines JavaScript and Vega-Lite. Therefore, not every line of code belongs to Vega-Lite; many parts are standard JavaScript used for data loading, transformation, and preparation.

## params/selection skeleton:

Selection (create)

Params (store)

Interaction (apply)

### selection:

Selection = capturing user action on chart

Types of Selection	Vega-Lite	Description	Deneb
Point Selection	<code>vl.selectPoint()</code>	Click on points	<pre>{ "select":   { "type": "point" } }</pre>
Interval Selection (Brush)	<code>vl.selectInterval()</code>	Drag to create a brush	<pre>{ "select":   { "type": "interval" } }</pre>

defining Named Selection to be able use it for combining two charts

E.g: `vl.selectPoint().name("click")`

### Params:

Params = container that stores selection state

Type	Example
Inline selection	<code>.params(vl.selectPoint())</code>
Named param	<code>const brush = vl.selectInterval().name("brush")</code>
Shared	<code>.params(brush)</code>

## Deneb

```

"params": [
{
"name": "brush",
"select": {
"type": "interval"
}
}
]

```

## interactions:

interaction=effect of selection on visualization, shows effect

## Types of Interaction:

interaction	vega-lite	Deneb
Filter interaction	.transform(vl.filter(click))	"filter":{"param":"click"}`
Zoom interaction	.scale({ domain: { param: "brush" } })	scale":{"domain":{"param":"brush"}}
Cross-filter	selectPoint + filter other chart	
Linked views	same brush used in 2 charts	

## one part of encoding :

### scale:

Scale maps data to space

Use scale when you want CONTROL over mapping:

1. Custom colors domain    range colors
2. Pie/donut angles value    angle
3. Zoom / brush interaction scale.domain({param:"brush"})
4. Size encoding value    radius

WHERE SCALE IS USED

ONLY INSIDE ENCODING CHANNELS encoding.x.scale(...)

Scale + Brush:..scale({ domain: { param: "brush" } }) means brush controls visible range

## one part of layout:

### Facet

Facet = split one dataset into multiple charts, when you want small multiples

E.g: `vl.facet().fieldN("Region")`

### Vcant

`vconcat` = stack separate charts vertically

E.g: `vl.vconcat(chart1, chart2)`

## Full Flow

1. Selection (user action)
2. Params (store selection)
3. Transform (filter data)
4. Encoding (map data → visuals)
5. Scale (position mapping)
6. Interaction (result behavior)
7. Layout (facet / vconcat)

to conclude : you can see one interactive visualization with two codes ( deneb & observable)"

 chart

deneb code	observable code
github	github...